YOLOS 家族架构详解

纯 ViT 目标检测 · 100 个 detection token · 位置编码可插值 + 中间层再注入 · 匈牙利匹配

依据：transformers 源码（本机 editable）E:\transformers\src\transformers\models\yolos\（版本 5.23.0.dev0，本文档所有行号/字段均以此为准），官方论文 arXiv:2106.00666（Fang et al., 2021, NeurIPS），HF 模型卡交叉验证。YOLOS（You Only Look at One Sequence）是华中科技大学 Fang 等提出的目标检测模型：不需要 CNN 主干、也不需要 DETR 的 encoder-decoder 跨注意力——一个普通 ViT encoder + 100 个可学习 detection token 就够了。标注「待确认」处未编造。

1 一句话定位

目标检测不需要 CNN 主干、也不需要 DETR 那套 encoder-decoder 跨注意力，一个普通 ViT encoder + 100 个可学习 detection token 就够了——把检测重新表述为「读一个 token 序列」，序列末尾的 detection token 直接输出集合式预测（分类 + 框回归）。历史意义：第一个「纯 ViT、零 CNN 归纳偏置」的检测器，用于验证 ViT 作为通用视觉 backbone 的边界。

2 源码模块与类关系

文件	职责
configuration_yolos.py	YolosConfig（单文件，无 text/vision 分拆）
modeling_yolos.py	全部模型类（视觉嵌入 → encoder → 检测头）
modular_yolos.py	模块化定义源，modeling_yolos.py 由它自动生成（文件头有警示）
image_processing_yolos.py / image_processing_pil_yolos.py	图像预处理 + post_process_object_detection（归一化框还原像素坐标）
convert_yolos_to_pytorch.py	官方权重 → HF 格式

表 1：YOLOS 源码文件分工

2.1 modeling_yolos.py 类清单

类	行号	职责
YolosObjectDetectionOutput	:43	检测输出 dataclass（loss/logits/pred_boxes/auxiliary_outputs）
YolosEmbeddings	:76	CLS + patch + detection token 拼接、位置编码、dropout
InterpolateInitialPositionEmbeddings	:118	初始位置编码按输入分辨率插值
InterpolateMidPositionEmbeddings	:147	中间层位置编码按分辨率插值
YolosPatchEmbeddings	:187	Conv2d 分块投影
YolosSelfAttention / YolosAttention	:250 / :324	自注意力（默认 eager，可切 SDPA/Flash/Flex）
YolosLayer	:371	单层 Transformer（继承 GradientCheckpointingLayer）
YolosEncoder	:405	12 层编码器 + 中间层位置编码逻辑（:419-427）
YolosModel	:451	backbone：输出 last_hidden_state + pooler
YolosMLPPredictionHead	:510	检测用 MLP 头
YolosForObjectDetection	:534	检测任务模型：ViT + 分类/框回归头

表 2：modeling_yolos.py 类清单

3 配置字段（YolosConfig，configuration_yolos.py:24）

model_type = "yolos"，默认 checkpoint 口径为 hustvl/yolos-base：

字段	默认值	说明
hidden_size	768	token 维度
num_hidden_layers	12	encoder 层数
num_attention_heads	12	注意力头数

字段	默认值	说明
intermediate_size	3072	FFN 中间维度
hidden_act	gelu	激活
image_size	(512, 864)	输入分辨率（高, 宽）
patch_size	16	分块大小 → 32×54=1728 个 patch
num_channels	3	输入通道
num_detection_tokens	100	可学习检测 token 数量
use_mid_position_embeddings	True	是否在中间层再加位置编码
auxiliary_loss	False	是否启用逐层辅助损失
class_cost / bbox_cost / giou_cost	1 / 5 / 2	匈牙利匹配代价系数
bbox_loss_coefficient / giou_loss_coefficient	5 / 2	损失加权系数
eos_coefficient	0.1	no-object 类损失权重

表 3：YolosConfig 配置字段

4 端到端架构总览

输入 512×864 → patch 16 → 1728 patch token；拼接 [CLS]×1 + patch×1728 + detection×100 = 1829 token 序列；12 层纯 ViT encoder（位置编码可插值 + 中间层再注入）；取末尾 100 个 detection token 过共享 MLP 头输出 N+1 类 logits 与归一化框。

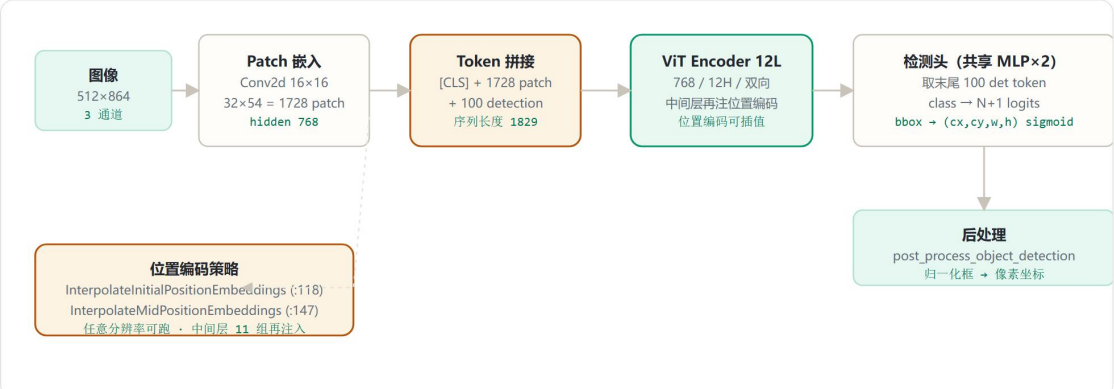


图 1：YOLOS 端到端架构总览

5 核心机制（对照代码）

5.1 Token 序列拼接（YolosEmbeddings :76）

`[CLS]×1 + patch_tokens×1728 + detection_tokens×100 → 序列长度 1829`

`# cls_token：1 个可学习参数（1×1×768）`

`# detection_tokens：100 个可学习参数（1×100×768），相当于 DETR 的 object queries，`

`# 但完全靠自注意力与 patch token 交互`

`# 位置编码 position_embeddings 形状为 (1, 1728+100+1, 768)，随分辨率插值后加性叠加`

5.2 位置编码：可插值 + 中间层再注入

初始位置编码（InterpolateInitialPositionEmbeddings :118）：forward 时按实际 (height, width) 把 [CLS] 位置、patch 位置、detection 位置拆开，patch 部分用双线性插值缩放到目标网格再拼回——因此任意分辨率输入都能跑。

中间层位置编码（InterpolateMidPositionEmbeddings :147；YolosEncoder :419-427）：当 use_mid_position_embeddings=True，除最后一层外，每层输出后再叠加一次插值后的位置编码（共 11 组）——这是 YOLOS 论文的重要技巧：弥补深层自注意力对位置信息稀释的问题。

5.3 Patch 嵌入（YolosPatchEmbeddings :187）

Conv2d(3, 768, kernel_size=16, stride=16)，num_patches = (864//16)×(512//16) = 54×32 = 1728。

5.4 检测头（YolosForObjectDetection :534）

1) self.vit = YolosModel(config, add_pooling_layer=False)（不加池化层）；2) 取 encoder 最后输出的末尾 100 个 detection token（:609）：sequence_output[:, -100:, :]；3) 两个共享 MLP 头（均为 3 层，input/hidden=768）：class_labels_classifier → num_labels + 1（含 no-object）；bbox_predictor → 4 维 (cx, cy, w, h)，sigmoid 归一化（:613）；4)

辅助损失（auxiliary_loss=True 且给 labels 时，:618-621）：把每层 encoder 的中间输出过同一对头，生成 auxiliary_outputs（对齐 DETR 的 aux loss 设计）。

6 训练与损失（DETR 式集合预测）

环节	要点
匈牙利二分匹配	训练时把预测框与 GT 框做最优匹配，匹配代价 = class_cost×分类 + bbox_cost×L1 + giou_cost×GIoU（1/5/2）
损失	匹配后的分类 CE + bbox L1 + GIoU，系数 5/2；eos_coefficient=0.1 压低 no-object 类对损失的影响
推理	YolosImageProcessor.post_process_object_detection 把归一化框还原为像素坐标（支持阈值过滤 + top-k）

表 4：训练与推理要点



图 2：匈牙利匹配集合预测流程

7 模型家族速览

检查点	规模档位	说明
hustvl/yolos-tiny	最小变体	~7M 级
hustvl/yolos-small	中等变体	—
hustvl/yolos-base	最大变体	ViT-Base 规模（hidden 768 / 12 层）

表 5：YOLOS 家族检查点

各 checkpoint 精确参数量与训练输入分辨率以官方模型卡为准（本机未下载权重验证，待确认）。

一分钟结论

维度	要点
架构	纯 ViT encoder（12L/768/12H）+ 100 个 detection token
输入	512×864，patch 16 → 1728 patch + CLS + 100 det = 1829 token
关键技巧	位置编码可插值；中间层再次注入位置编码
输出	每 token：N+1 类 logits + (cx,cy,w,h) 归一化框
损失	匈牙利匹配 + CE + L1 + GIoU（1/5/2，eos 0.1）
代码入口	YolosModel（backbone）/ YolosForObjectDetection（检测）

表 6：一分钟结论

与检测/多模态演进的关系：DETR 用 encoder-decoder + cross-attention 做集合预测；YOLOS 只用 encoder，detection token 通过与 patch token 的全自注意力完成信息聚合，跨注意力并非必需。后续 ViTDet、DINO 等进一步证明 ViT backbone 在检测上的潜力。「位置编码随分辨率插值」与 Qwen2-VL / SigLIP-2 NaFlex 的可变分辨率思想同源（都是让 ViT 摆脱固定分辨率输入）。

8 来源

[1] 论文：You Only Look at One Sequence: Rethinking Transformer in Vision through Object Detection，arXiv:2106.00666（Fang et al., 2021, NeurIPS）

[2] HF 模型卡：hustvl/yolos-base / hustvl/yolos-tiny

[3] transformers 源码（本机 editable）：src/transformers/models/yolos/（5.23.0.dev0）

待确认/边界：各 checkpoint 精确参数量与训练分辨率以官方模型卡为准（本机未下载权重验证）；YOLOS 在 transformers 中只有一个模块版本，无后续系列（YOLOS-X 等未收录）。